Python Engineering - Week 01 / Class 04

EX11 - Taylor e^x como funcion reutilizable

De un loop que funciona a una herramienta matematica reutilizable

Ruta actual: 01_python_bootcamp / week_01_logic / class_04_functions / exercises / ex11_taylor_exp_function.py

1. Objetivo del ejercicio

En EX07 construimos la aproximacion de Maclaurin para e^x con un loop. En EX11 no vamos a aprender Taylor de nuevo: vamos a **encapsular ese algoritmo dentro de una funcion** para poder usarlo muchas veces sin repetir el codigo.

e^x ~= sum_(n=0)^N x^n / n!

Queremos pasar de un script lineal a una interfaz sencilla:

```
approximation = taylor_exp(x, order)
```

Mentalmente, la funcion se puede ver como una maquina matematica:

(x, order) -> taylor_exp(...) -> e_N(x)

2. Que ocurre dentro de la funcion

Cada llamada crea su propio **scope local**. Las variables del algoritmo - por ejemplo `term`, `factorial` y `approximation` - pertenecen a esa llamada y no necesitan existir como variables globales.

```
def taylor_exp(x, order):
    approximation = 0.0
    factorial = 1

    for n in range(0, order + 1):
        term = x**n / factorial
        approximation += term
        factorial *= n + 1

    return approximation
```

La idea clave es que **factorial transporta estado entre iteraciones**. Al empezar la iteracion n, queremos que contenga n!. Al terminar, lo preparamos para la siguiente iteracion.

n	factorial al entrar	term	factorial preparado
0	1 = 0!	1	1 = 1!
1	1 = 1!	x	2 = 2!
2	2 = 2!	x^2 / 2	6 = 3!
3	6 = 3!	x^3 / 6	24 = 4!

3. Parametros, argumentos y reutilizacion

La definicion usa **parametros**:

```
def taylor_exp(x, order):
```

y cada llamada suministra **argumentos**:

```
a = taylor_exp(0.5, 5)
b = taylor_exp(0.5, 10)
c = taylor_exp(2.0, 10)
```

Las tres llamadas ejecutan el mismo algoritmo, pero cada una tiene su propio conjunto de variables locales. Esto es exactamente la ventaja de encapsular el calculo.

4. Validacion que debe hacer la funcion

Para este ejercicio, `order` representa el mayor orden polinomico incluido. Por tanto debe ser un entero no negativo:

order in Z, order >= 0

Mantendremos el patron aprendido en EX09 y EX10: si la entrada no es valida, la funcion puede terminar con `return None`. El programa principal decide despues si imprime o no el resultado.

No necesitas una rama especial para order = 0: el loop general ya genera el termino x^0 / 0! = 1.

5. Docstring: documentar la funcion donde vive

Hasta ahora documentabamos el archivo. A partir de esta clase empezaremos tambien a documentar cada funcion con un **docstring**, escrito justo debajo de `def`:

```
def taylor_exp(x, order):
    """
    Approximate e^x using the Maclaurin series.

    Parameters
    ----------
    x : float
        Point where the exponential is evaluated.
    order : int
        Highest polynomial order included.

    Returns
    -------
    float or None
        Taylor approximation, or None for invalid order.
    """
```

Un docstring no cambia el calculo; sirve para que la funcion explique por si misma que recibe, que hace y que devuelve.

6. Type hints basicos (opcional en EX11)

Podemos anotar el tipo esperado sin obligar a Python a cumplirlo. Para Python 3.9, una forma compatible es:

```
from typing import Optional

def taylor_exp(x: float, order: int) -> Optional[float]:
    ...
```

Importante: los type hints son documentacion y ayuda para editores/analizadores; Python no convierte automaticamente un valor incorrecto ni impide por si solo una llamada.

7. Que debe quedar dentro y fuera

Dentro de la funcion: validacion del orden, loop de Taylor, acumuladores y return.

Fuera de la funcion: valores concretos de prueba, comparacion contra math.exp(), calculo de errores y prints.

```
x_value = 0.5
order = 5

approx = taylor_exp(x_value, order)

if approx is not None:
    exact = exp(x_value)
    absolute_error = abs(exact - approx)
    print(...)
```

Esto separa el **motor matematico** de la **presentacion y validacion del experimento**.

8. Casos de prueba obligatorios

Caso	x	order	Que comprobar
A	0.5	0	resultado = 1
B	0.5	5	aproximacion razonable
C	0.5	10	menor error que order=5
D	2.0	10	reutilizacion con otro x
E	1.0	-1	return None
F	1.0	3.5	return None

9. Lo que NO quiero todavia

No uses scipy, numpy.exp ni una funcion ya hecha para generar la aproximacion. `math.exp()` se usa solo como referencia independiente para validar tu funcion.

Tampoco necesitamos SymPy en esta version: ahora buscamos una funcion **numerica y reutilizable** que reciba un x concreto y devuelva un numero.

10. Learning Tracker - EX11

Concepto	Objetivo
Funcion con loop interno	Consolidar encapsulacion
Scope local	Evitar depender de globales
return	Entregar el resultado al caller
Docstring	Documentar la interfaz
Type hints	Introduccion opcional

Reutilizacion	Variar x y order sin copiar codigo
Validacion independiente	Comparar con math.exp()

Mision: crea `ex11_taylor_exp_function.py`. Quiero ver una funcion autocontenida, sin depender de x, N, factorial o acumuladores globales, y varias llamadas desde el programa principal.